MISSION 01 · DAY 1

Git & Version Control

Every codebase, pipeline, and infrastructure config you will ever build lives in Git.

96%	330M+	Day 1	Non-neg.
of dev teams use Git	repos on GitHub	tool at every tech company	skill for data engineers

Topics: Why Git · Core Concepts · Workflow · Commands · Branching · Merge & Rebase · GitHub · Git for DE

Today's Agenda

01	Why Version Control	Context
02	Git vs GitHub	Concepts
03	Core Concepts	Concepts
04	The Git Workflow	Concepts
05	Essential Commands	Hands-on
06	Branching Strategy	Hands-on
07	Merge vs Rebase	Deep Dive
08	Merge Conflicts	Deep Dive
09	.gitignore & Best Practices	Deep Dive
10	Git in Data Engineering	Applied
11	GitHub Collaboration	Applied
12	Git at Sigma DataTech	Mission

Why Version Control Exists

CONTEXT

The Problem

- Concurrent edits cause silent overwrites and data loss
- No audit trail — changes are unattributable and irreversible
- No rollback — no recovery strategy

What Git Gives You

- Immutable history — every file, every point in time
- Atomic commits — authored, timestamped, intentional
- Branching — parallel workstreams without interference
- Distributed — every clone is a full backup

Real-World Usage

Data pipelines
Pipeline logic versioned alongside schema changes

Infrastructure as Code
Terraform, CloudFormation, Ansible — all live in Git

Application code
Every commit reviewed, tested, deployed

ML workflows
Model configs, training scripts, experiment tracking

Git vs GitHub — Not the Same Thing

CONCEPTS

>_ Git

- Distributed version control system
- Runs locally on your machine
- Open source — created by Linus Torvalds (2005)
- Tracks file changes and history
- Works completely offline
- The engine — all logic lives here

VS

GitHub

- Cloud hosting platform for Git repositories
- Adds collaboration layer on top of Git
- Pull Requests, Issues, Actions, Projects
- Owned by Microsoft (acquired 2018)
- Alternatives: GitLab, Bitbucket, Azure DevOps
- The interface — Git is still doing the work

Core Concepts

Repository

The project folder Git is tracking — contains all files and their complete history

Commit

A snapshot of changes — immutable, authored, timestamped, with a message

Branch

An independent line of development — isolates work without affecting main

Merge

Combines changes from one branch into another — creates a merge commit

Remote

A version of the repo hosted on a server (GitHub) — teams sync via push / pull

Working Tree

Your local files as they currently exist — what you see in your editor

The Git Workflow

CONCEPTS

Working Directory

Files you edit

→ git add

Staging Area

Changes queued for commit

→ git commit

Local Repository

Committed history on your disk

→ git push

Remote Repository

Shared on GitHub / server

← *git pull / git fetch*

Staging is intentional — you choose exactly what goes into each commit. Not all changed files need to be committed together.

Essential Commands

COMMAND	WHAT IT DOES
`git init`	Initialise a new Git repository in current directory
`git clone <url>`	Copy a remote repository to your local machine
`git status`	Show changed files — staged, unstaged, untracked
`git add <file>`	Stage a file for the next commit
`git add .`	Stage all changed files in current directory
`git commit -m "msg"`	Save staged changes as a commit with a message
`git push origin main`	Push local commits to the remote repository
`git pull`	Fetch remote changes and merge into current branch
`git log --oneline`	View commit history in compact format
`git diff`	Show unstaged changes line by line

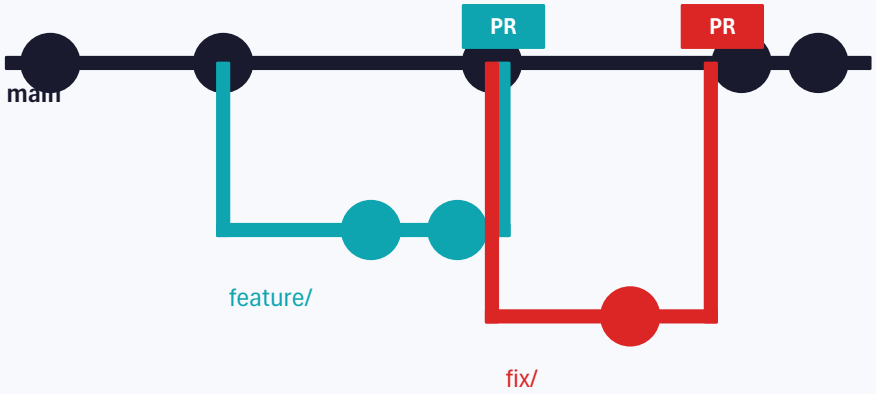
Branching Strategy

HANDS-ON

Why Branch?

- Isolate features — no impact on production code
- Multiple engineers work in parallel without conflicts
- Easy to discard failed experiments
- Every PR maps to exactly one branch

Typical Branch Flow



Naming Conventions

feature/	feature/user-auth
fix/	fix/null-pointer-pipeline
hotfix/	hotfix/prod-data-loss
release/	release/v2.1.0

Rule: Never commit directly to main. Always branch → commit → PR → review → merge.

Merge vs Rebase

DEEP DIVE

git merge

- Joins two branches — creates a merge commit
- Preserves full history exactly as it happened
- History can become non-linear
- Visible record of when branches diverged

✓ **Use for: merging feature branches to main**

- git merge feature/my-branch

git rebase

- Replays commits on top of another branch
- Creates a linear, clean commit history
- Rewrites commit hashes — changes history
- Harder to trace when changes were integrated

✓ **Use for: cleaning up local commits before PR**

- git rebase main

Golden Rule: Never rebase a branch that others are working on. Rebase is for local cleanup only.

Resolving Merge Conflicts

DEEP DIVE

What Causes a Conflict?

- Two branches modify the same lines of the same file
- A file is deleted in one branch but modified in another
- Git cannot decide which version to keep — it asks you

What a Conflict Looks Like

```
<<<<<<< HEAD
 load_data('v2.csv')
=======
 load_data('v3.csv')
>>>>>>> feature/update
```

Resolution Steps

1. Run git status — lists all conflicted files
2. Open each file — find the conflict markers
3. Edit the file — keep what is correct
4. Remove all <<< === >>> marker lines
5. git add the resolved file
6. git commit to complete the merge

Pro Tip: VS Code shows conflicts with inline buttons — Accept Current, Accept Incoming, Accept Both. Use them. It is faster than editing markers manually.

.gitignore & Commit Best Practices

DEEP DIVE

.gitignore — What to Exclude

```
# Python
__pycache__/
*.pyc

# Environment
.env
venv/

# Data files
*.csv
*.parquet

# IDE
.vscode/
.idea/
```

Commit Best Practices

DO

- One logical change per commit
- Imperative message: Add, Fix, Update, Remove
- Reference ticket numbers where relevant
- Commit working code — never break the build

DON'T

- "WIP", "stuff", "fix" as commit messages
- Committing secrets, credentials, API keys
- Mixing unrelated changes in one commit
- Committing compiled files or binaries

Git in Data Engineering

Pipeline Code

- ETL scripts, DAGs, transformation logic
- Every pipeline change is a PR — reviewable
- Roll back a broken pipeline in 60 seconds

Infrastructure as Code

- Terraform, CloudFormation, Ansible all in Git
- Environment configs tracked alongside code
- Drift detection — what changed vs last deploy

ML & Experiment Tracking

- Model training configs versioned per experiment
- Reproduce any result — code + config + data ref
- Branching per experiment — compare approaches

Schema & Migration Management

- SQL migration files committed in order
- Schema changes reviewed before applying to prod
- History of every DDL change ever made

GitHub Collaboration

APPLIED

The Pull Request Flow

1. Fork / Clone
2. Create Branch
3. Commit Changes
4. Push Branch
5. Open Pull Request
6. Review & Merge

Key Concepts

Pull Request
Proposal to merge code — triggers review process before anything reaches main

Code Review
Teammates inspect changes — catch bugs, improve quality, share knowledge

Fork
Your personal copy of a repo — make changes without affecting the original

GitHub Actions
CI/CD — automate tests, linting, and deployments on every push or PR

Issues
Track bugs and features — link commits and PRs to issues for traceability

Git at Sigma DataTech

MISSION

Your Repository Structure — Today

```
sigma-datatech/
  ├── pipelines/
  │    ├── bronze/
  │    ├── silver/
  │    └── gold/
  ├── infra/
  │    └── terraform/
  ├── notebooks/
  ├── tests/
  └── README.md
```

Team Git Rules

- 🔒 main is protected — no direct commits
- 🌿 Branch per mission: mission/01-git
- 📝 PR title format: [M01] Add pipeline logic
- 👀 Minimum 1 reviewer before merge
- ✅ Linked issue for every PR
- 🚫 .env and *.csv always in .gitignore

MISSION 01 · COMPLETE

Git is live at Sigma DataTech.

What the team can now do:

- ✓ Version every file in the codebase
- ✓ Branch and work in parallel without conflict
- ✓ Review each other's code before merge
- ✓ Roll back any breaking change in seconds